Scope and Persistence

Where We Left Off

Functions *have parameters and local variables*

Loops *declare loop variables like i*

Some names vanish at the closing brace

Some values survive between calls

Today: the rules behind both

Two Questions for Every Variable

Scope: *where can the* **name** *be used?*

a property of the source code

Persistence: *how long does the* **memory** *exist?*

a property of the running program

Also called **visibility** *and* **lifetime**

Names Are Addresses

A **variable** *is a symbolic name for an address in the* **data** *section of the program*

A **function** *is a symbolic name for an address in the* **instruction** *section (the text)*

You must be inside the **scope** *to see the address as a symbol*

Lexical Scoping

Scope defines where a variable or a function name is **valid**

A variable cannot be referenced by name from **outside** *its scope*

"Lexical" means it is decided by where the text appears in the source file

Scopes Are Braces

Lexical scopes are defined with **{ }**

```
{                       outer scope
    {                   inner scope
    }                   inner names gone
}                       outer names gone
```

The same rule in C, C++, and Rust

Global Scope

The scope **outside of all functions**

C, C++: *int stepG ;*
Rust: *static STEP_G : i32 = 0 ;*

A variable defined there is a **global** *variable*

My suggestion: use capital **G** *suffix to marks it as global*

Function Scope

The scope of a function's **parameters** *is the complete body of the function*

Local variables are in scope from their declaration to the closing brace

At the close of the scope, the symbol **disappears**

Scope Example in C (1 of 2)

```
void p ( char x )
{
        /* p, x */
        char y ;
        /* p, x, y */
        char zoo ;
        /* p, x, y, zoo */
}
/* p */
double expG ;
/* p, expG */
```

Comments show symbols which are currently visible

Scope Example in C (2 of 2)

```
void qsort ( char *s1, char *s2 )
{
        /* p, expG, qsort, s1, s2 */
        char b ;
        {
                int count ;
                /* p, expG, qsort, s1, s2, b, count */
        }
        /* p, expG, qsort, s1, s2, b */
}
/* p, expG, qsort */
```

The Same Idea in Rust

```
fn p ( x : u8 ) {
        // p, x
        let y : u8 = 0 ;
        // p, x, y
        {
                let count = 0 ;                 // p, x, y, count
        }
        // p, x, y
}
```

Order Matters in C, Not in Rust

C, C++: a name is visible from its declaration **downward** *to the end of its scope*

Rust: functions, statics, and types are visible in the **whole module**, *even above them*

Rust parses the entire file before dataflow analysis like Verilog

Rust local variables still follow top-to-bottom order

Shadowing

An inner scope may reuse an outer name:

```
int x = 5 ;
{  int x = 10 ;  }                    inner x hides outer x
```

Rust also allows it in the **same** *scope:*

```
let x = 5 ;
let x = x * 2 ;                       a new x, now 10
```

Asking for trouble

Hard to understand if not author

Scope and Loops

for (int i = 0 ; i < n ; i++) { ... }

i is gone here

for i in 0..n { ... }

i is gone here too

A loop variable's scope is the loop

But Where Is the Memory?

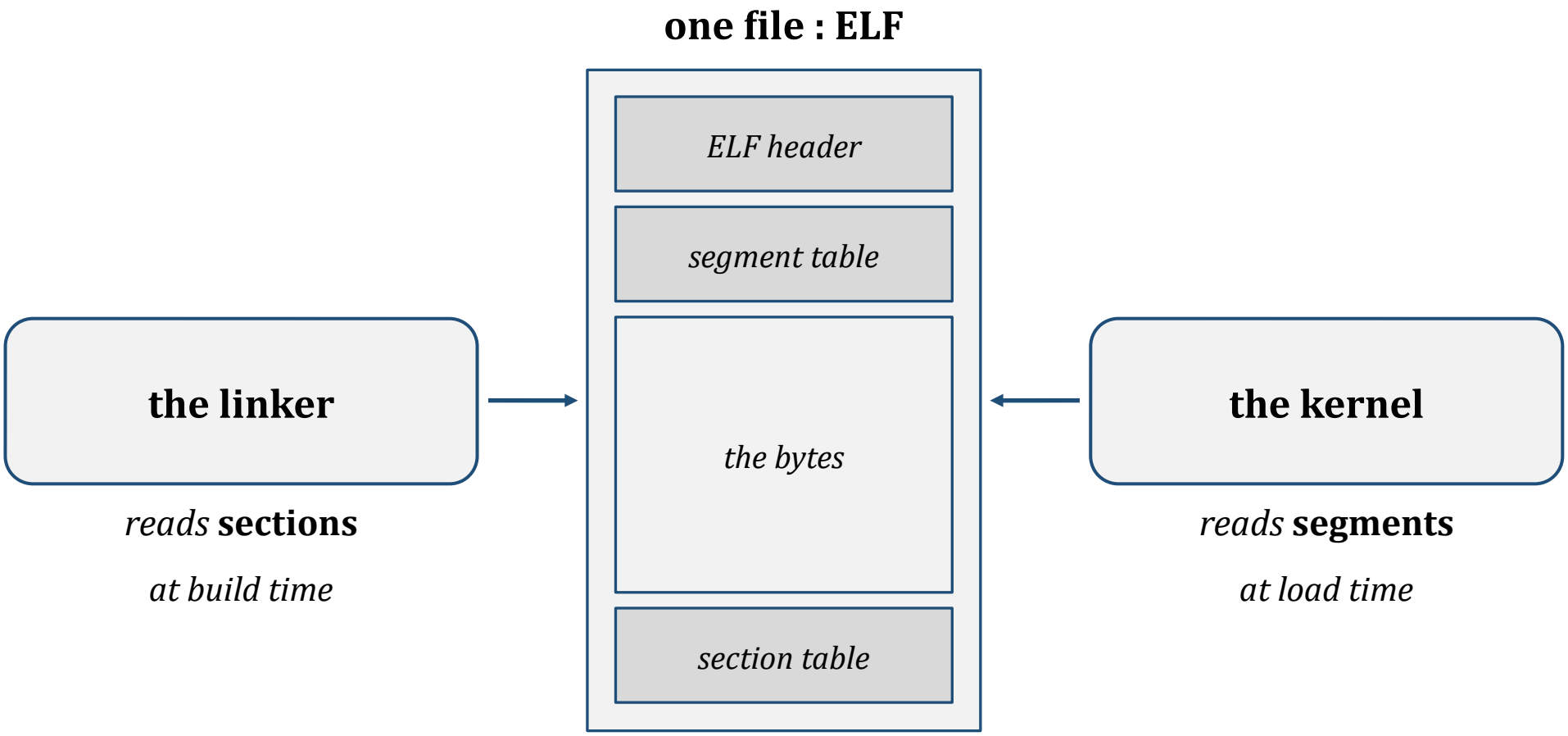
A variable definition allocates memory

Where it goes depends on the variable's **scope** *and its* **lifetime**

Three places memory can live:

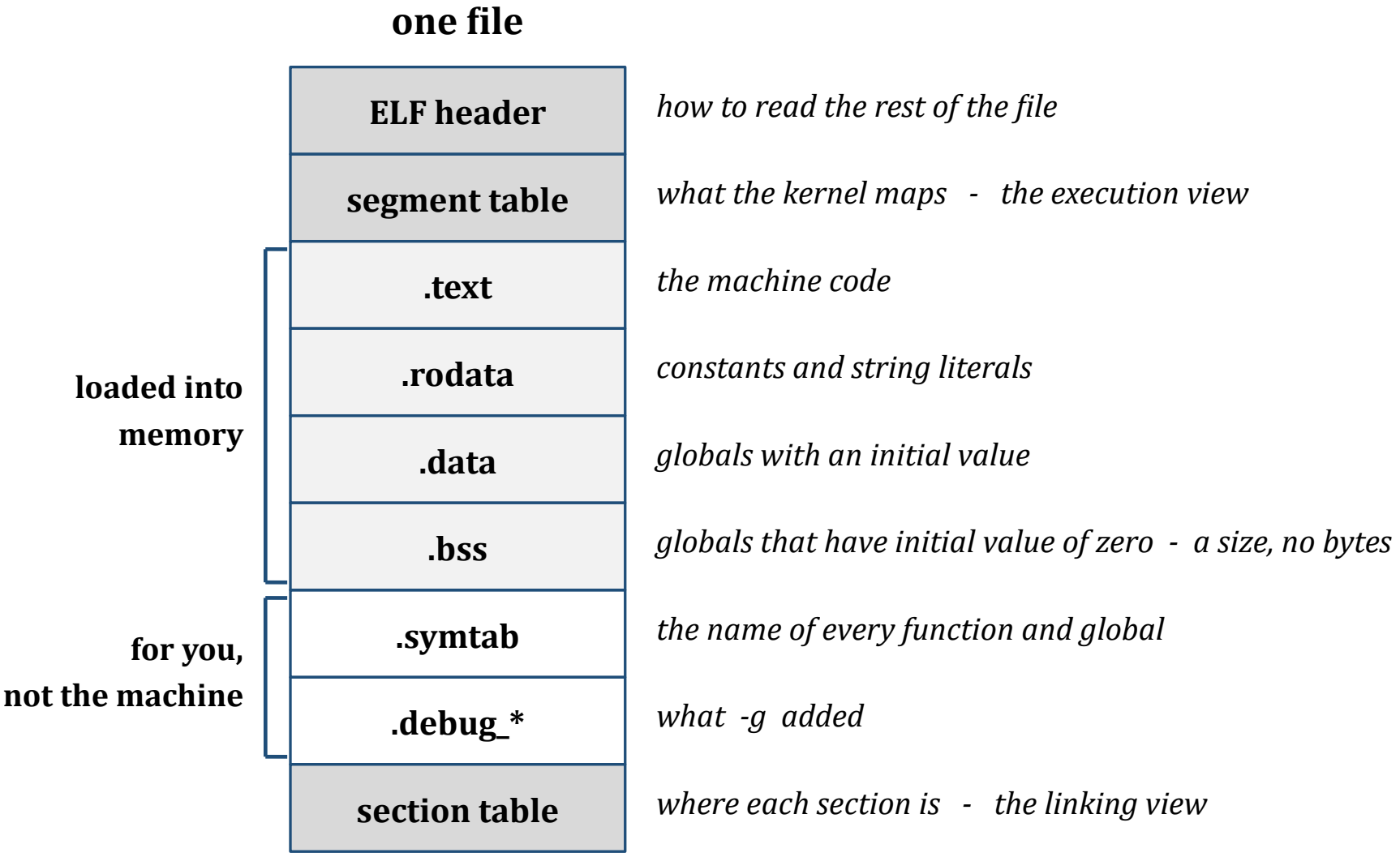
data segment *(of the program's object text)*
stack
heap

Two Views of Program File



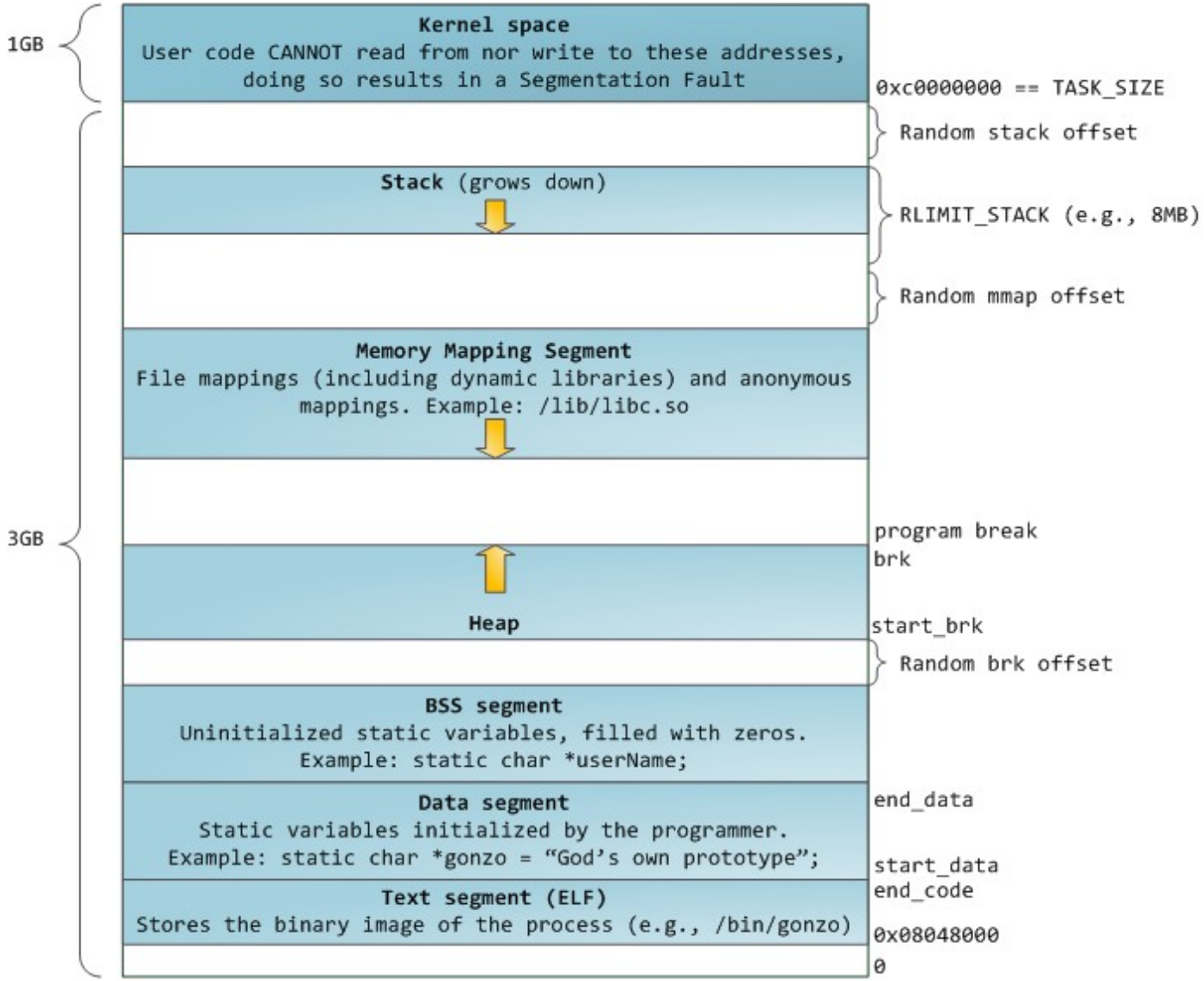
The same bytes, **described twice**, *for two different readers*

Inside the Program File



Two tables describe the same bytes - one for the **linker**, *one for the* **kernel**

Memory Map of a Linux Process



The program gets copied or "loaded" into the address space

Region 1: The Data Segment

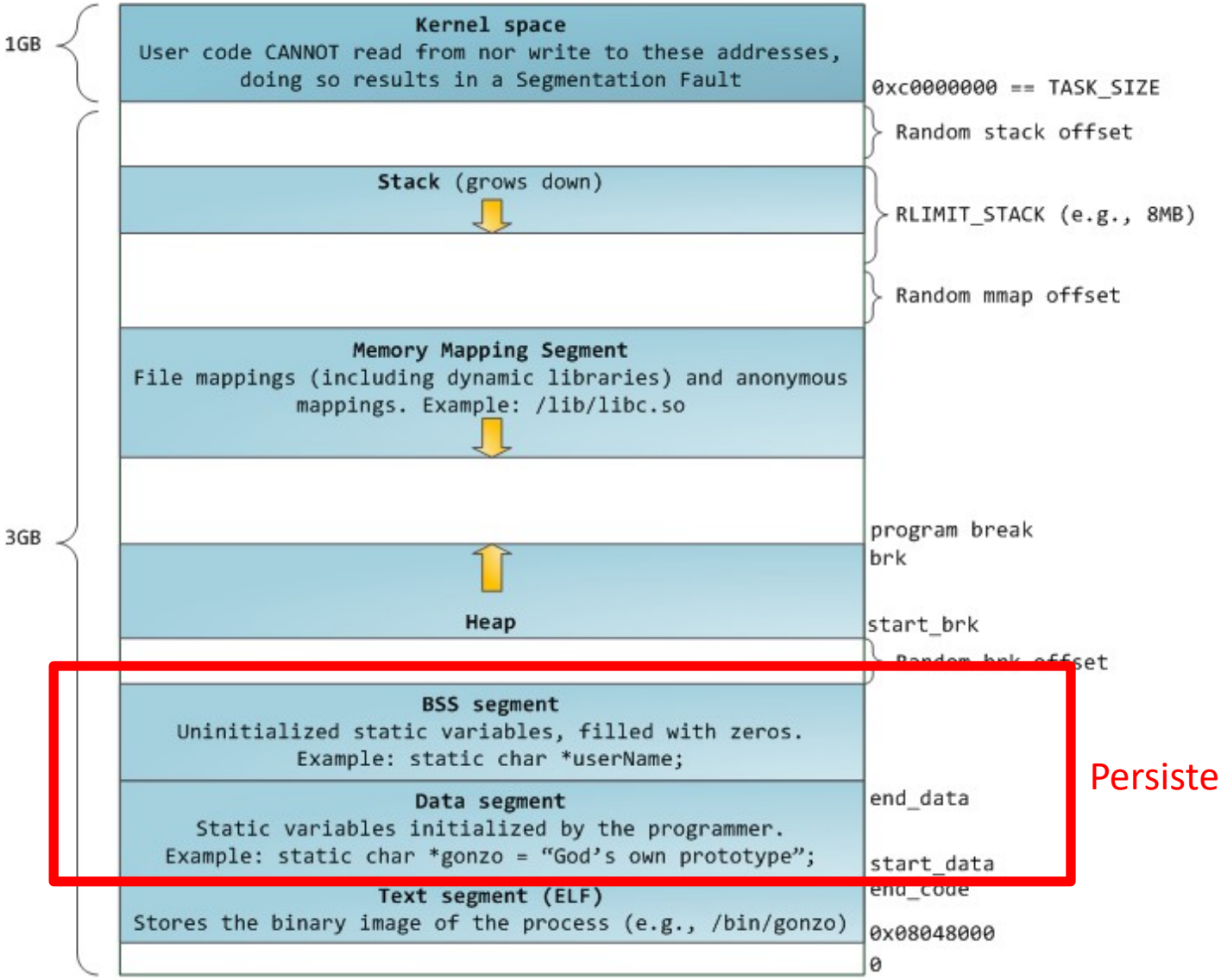
Persistent *variables live here*

Persistent mean lives the entire life of the program

Size known at **compile time** *- part of the object code*

Data*:* *variables with an initial value*
BSS*:* *variables with initial value of zero*
set to **zero** *when the program loads*
run length encoded
expanded during program load

Persistent Memory Mapping



The program gets copied or "loaded" into the address space

Region 2: The Stack

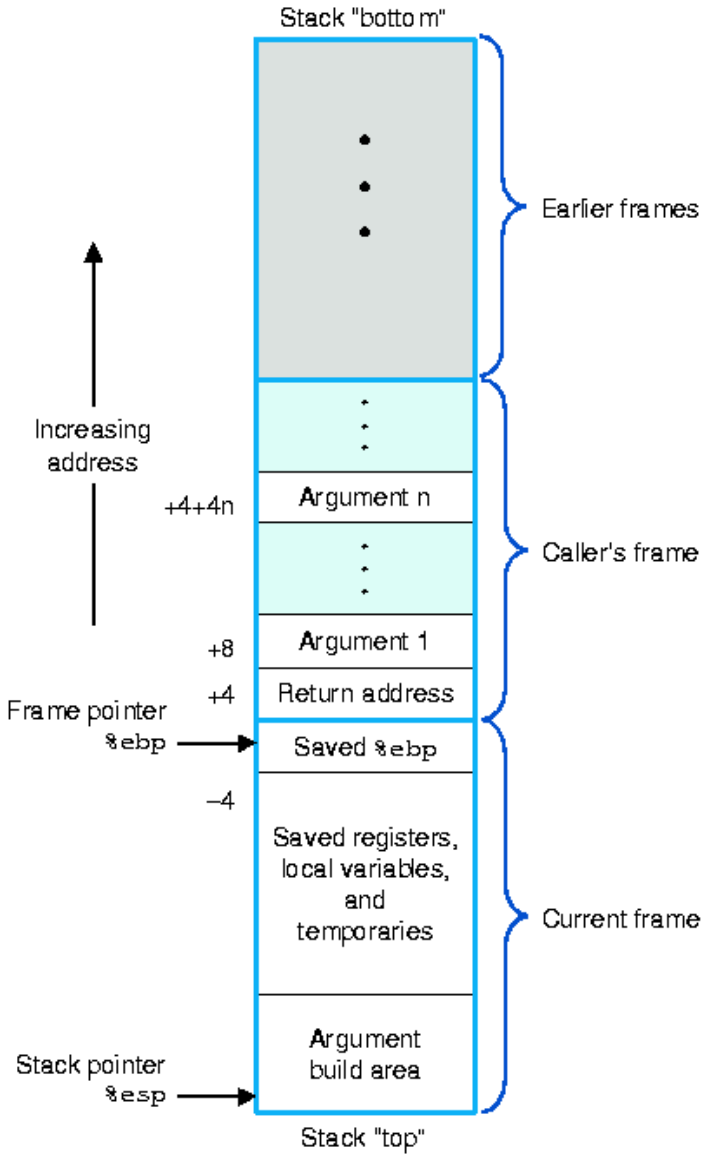
Holds each call's **arguments** *and* **local** *variables*

A **stack** *data structure: push on call, pop on return*

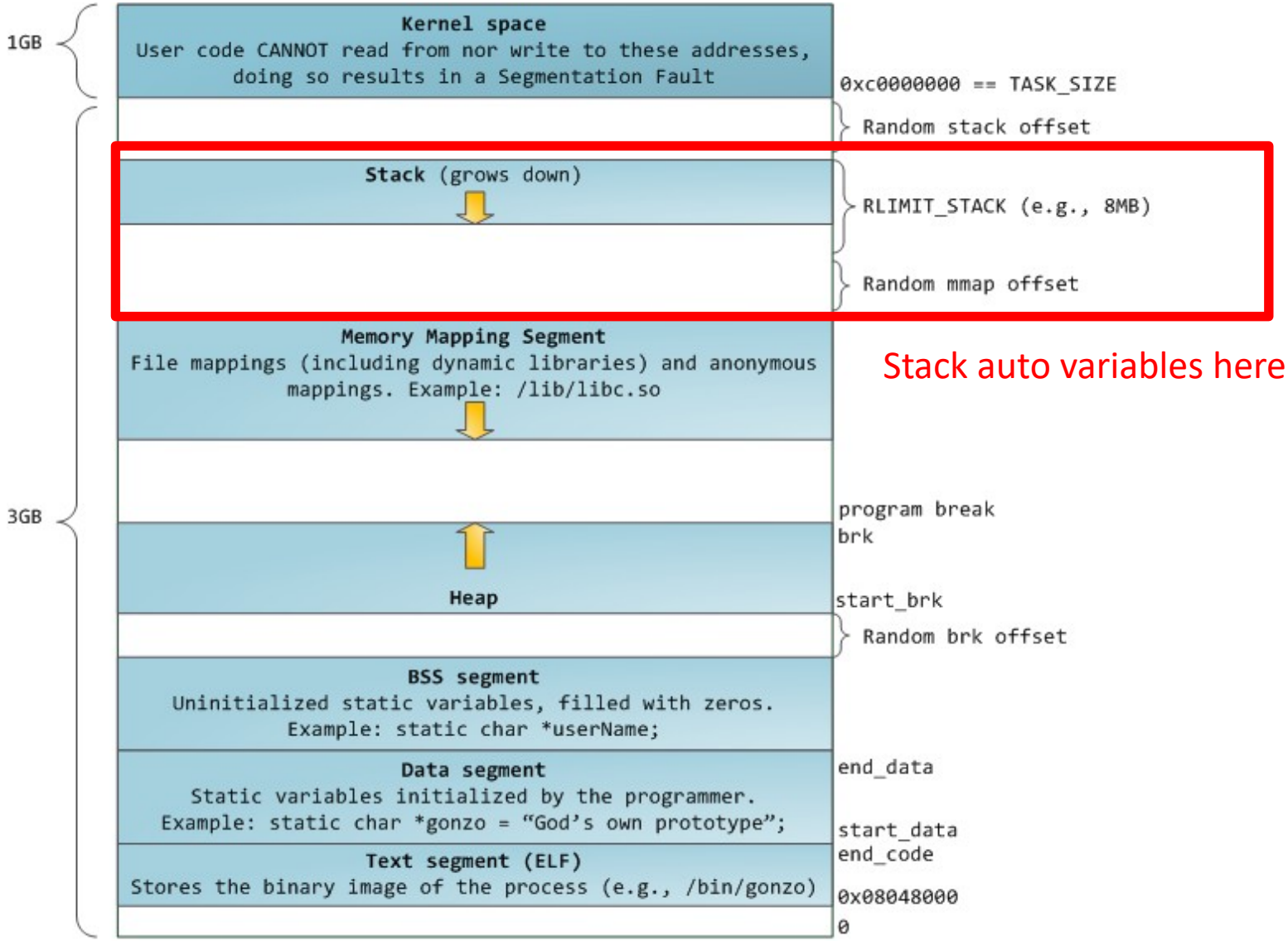
Located by the **stack pointer** *and* **frame pointer**

Memory on the stack is **uninitialized** *and* **temporary**

A Stack Frame



Stack Memory Mapping



The program gets copied or "loaded" into the address space

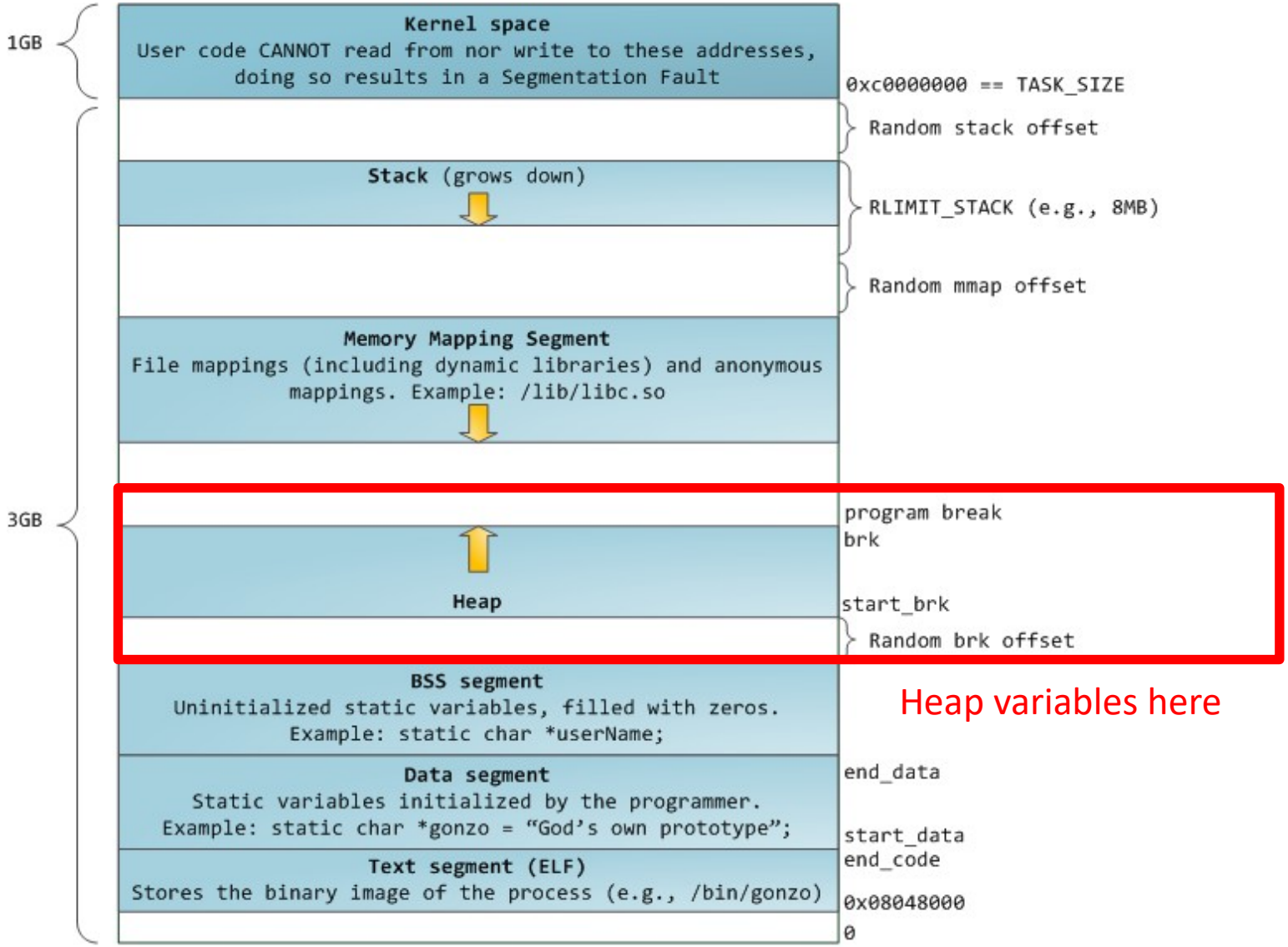
Region 3: The Heap

Dynamic *memory - size decided at run time*

The **program** *decides when it is created and when it is released*

Lives until it is freed - not tied to a function call

Heap Memory Mapping



The program gets copied or "loaded" into the address space

The Heap in C

```
float *buf = malloc ( 100 * sizeof ( float ) ) ;
if ( !(buf) )  { /* out of memory */ }

...  use buf[0] through buf[99]  ...

free ( buf ) ;
```

Use calloc to initialize memory to zero

Use realloc to resize the memory

Old area copied but new extra area uninitialized

The Heap in C++

*float *buf = new float[100] ;*

…

delete[] buf ;

Better: let a type own the memory

std::vector<float> buf (100) ;

auto p = std::make_unique<sample_t> ( ) ;

Freed automatically at the closing brace

The Heap in Rust

let buf = vec![0.0f32 ; 100] ;
let s = Box::new (sample) ;

No free. No delete.

When the **owner** *goes out of scope, the heap memory is released*

Tries to minimize the burden on programmer

Heap Bugs in C

Leak: *forget to free - memory is never returned*

Use after free: *read memory already given back*

Double free: *free the same memory twice*

C compiles all three without complaint

Rust rejects all three at compile time

Automatic Variables

Declared inside a function

Created on the **stack** *when the call starts*

Gone when the call returns

Called **auto** *variables in C: automatic allocation and release*

Auto Variables Start as Garbage

```
void my_function ( int expand )
{
        int step ;
        step = step + expand ;          bug
}
```

step was never initialized

C, C++: the program **does not crash** *– it silently uses whatever was on the stack*

Rust Refuses to Compile It

```
fn my_function ( expand : i32 ) {
        let step : i32 ;
        let total = step + expand ;
}
```

error: used binding step isn't initialized

Rust checks every path before any use

Persistent Variables

Their memory exists for the **entire** *program execution*

Stored in the **data segment**

Initialized to **zero** *or to a value you give,* **once**, *before main runs*

Persistent Variable Visibility

The visibility of a persistent variable is determined by its **scope***:*

inside a function	**local static**
outside, file only	**file static**
outside, all files	**global**

Same lifetime - three different scopes or visibilities

Local Static

```
void my_function ( int expand )
{
    static int stepL ;
    stepL = stepL + expand ;
}
```

Visible only **inside** *my_function*

But the value **persists** *between calls*

Example: Counting Calls

```
int count_calls ( void ) {
    static int countL = 0 ; // assignment is not needed!
    countL = countL + 1 ;
    return countL ;
}
```

Returns 1, then 2, then 3, ...

The = 0 happens **once**, *not on every call*

Local Static in Rust

```
use std::sync::atomic::{ AtomicU32, Ordering } ;

fn count_calls ( ) -> u32 {
        static COUNT_L : AtomicU32 = AtomicU32::new ( 0 ) ;
        COUNT_L.fetch_add ( 1, Ordering::Relaxed ) + 1
}
```

A mutable static must be made **safe to share**

Declarations

Used to reference a variable or a function **before** *it is defined*

Say what it **is** *- its type*

Apply to functions (instruction text) and to variables (memory addresses)

Definitions

Describe the **implementation** *of a function or variable*

Function definition: the parameters and the statements that implement it

Variable definition: describes **and allocates** *space for the address*

Function Declaration

Known as a **prototype**

extern <return_type> <name> (arglist) ;

*extern int access_data (char *array, int idx, int num_elements) ;*

Takes a pointer, an index, and a count

Returns an int

extern on Functions

extern *extends the visibility of a function to other files*

Not strictly necessary for functions, but recommended

Many **identical** *declarations are allowed*

Only **one** *definition*

Function Definition

```
int access_data ( char *array, int idx,  int num_elements ) {
        if ( ( idx >= 0 ) && ( idx < num_elements ) ) {
                return array[idx] ;
        }
        return -1 ;                     // out of bounds
}
```

This becomes **text***: machine instructions*

extern on Variables

extern double myNumG ;

Describes the **type** *but does* **not** *allocate*

A placeholder for the real address

The address is **resolved** *at link time*

myNumG is 8 bytes of double - defined elsewhere

Variable Definition

double myNumG ;

myNumG is the starting address of 8 bytes holding a double

This statement **allocates** *the memory*

Exactly **one** *file may contain this line*

Declarations in C++

The same rules as C: prototypes, extern

Declarations go in a **header** *(.h / .hpp)*
Definitions go in a **source** *file (.cpp)*

Break the **One Definition Rule** *and the linker reports a duplicate symbol*

Declarations in Rust

Rust has **no** *separate declarations and* **no** *header files*

mod sensor ;	*load sensor.rs as a module*
use sensor::adc_to_psi ;	*bring the name into scope*

The compiler sees all files of a Rust compilation at once

Global Variables

```
int stepG ;

int main ( int argc, char *argv[] )
{
      ...
}
```

Four bytes in the **data** *section, for the* **lifetime** *of the program*

Using a Global from Another File

another.c:

```
extern int stepG ;

void reset ( void ) {
    stepG = 10 ;
}
```

Usable in any scope and file that declares it

Globals in C++

Same as C, plus **namespaces***:*

```
namespace sensor {
    int stepG ;
}

sensor::stepG = 10 ;
```

A namespace keeps global names from colliding

Globals in Rust

static mut STEP_G : i32 = 0 ; // unlike C/C++ must set to zero

unsafe { STEP_G = 10 ; }

Every read or write of a **mutable** *global needs* **unsafe**

Rust is telling you global variables are risky

Safe Globals in Rust

```
use std::sync::atomic::{ AtomicI32, Ordering } ;

static STEP_G : AtomicI32 = AtomicI32::new ( 0 ) ;

STEP_G.store ( 10, Ordering::Relaxed ) ;
let s = STEP_G.load ( Ordering::Relaxed ) ;
```

No unsafe - the type makes sharing safe

Rust shows that your are explicitly loading the global before you use it!

Limiting Persistent Visibility

What if a persistent variable should be visible to **one file** *only?*

Use a **static** *definition:*

static int stepS ;

What is its scope?

File Scope: static

Defined outside any function
Scope: every function that **follows** *in the file*

```
static int stepS ;

void my_function ( int expand ) {
        stepS = expand ;
}
```

Other files cannot reach it, even with extern

File Scope in C++

static works as in C

Preferred: an **unnamed namespace**

```
namespace {
    int stepS ;
}
```

Everything inside is private to this file

Module Scope in Rust

Everything is **private** *to its module unless marked* **pub**

static STEP_S : i32 = 0 ;	*this module only*
pub static STEP_G : i32 = 0 ;	*visible to other modules*

Rust's default is C's static

Limiting Function Visibility

A function can be private to its file too

Declaration:

```
static <return_type> <name> ( arglist ) ;
```

Definition:

```
static <return_type> <name> ( arglist ) {
    ...
}
```

A Private Function

file: compute.c

```
static int my_static_func ( int x ) {
        return ( x * x ) - 1 ;
}
```

This function only exists in **compute.o**

No other file can call it

Private Functions in C++ and Rust

C++:

static or an unnamed namespace
private: member functions of a class

Rust:

fn helper ( )	*private to the module*
pub fn api ( )	*visible to other modules*
pub(crate) fn tool ( )	*visible in this* **crate** *only*

What Is a Crate?

The **unit of compilation** *in Rust*

One crate = one run of rustc

Produces either an **executable** *or a* **library** *other crates can use*

A crate may be one file or hundreds

Rust sees all the files of a crate at once

Global and Static: main.c

```
#include <stdio.h>
extern void my_sub ( void ) ;
int countG ;                                global definition
static int resetS ;                         static definition

int main ( int argc, char *argv[] ) {
        countG = 1 ;
        resetS = 1 ;
        my_sub ( ) ;
        fprintf ( stderr, "countG is %d not 1\n", countG ) ;
        fprintf ( stderr, "reset is still %d\n", resetS ) ;
        return 0 ;
}
```

Global and Static: mysub.c

```
#include <stdio.h>
extern int countG ;                    global reference
static int resetS ;                    a different resetS

void my_sub ( void ) {
        countG = 2 ;
        resetS = 0 ;
        fprintf ( stderr, "countG in my_sub is %d\n", countG ) ;
        fprintf ( stderr, "resetS in my_sub is %d\n", resetS ) ;
}
```

The Symbol Tables

gcc -c main.c ; nm main.o

C _countG
T _main
U _my_sub
s _resetS

gcc -c mysub.c ; nm mysub.o

U _countG
T _my_sub
s _resetS

T *text* **U** *undefined - linker must find it*
C *global data* **s** *lowercase = file private*

After Linking

gcc -o myprog main.o mysub.o ; nm myprog

S _countG	**one** *common copy*
s _resetS	
s _resetS	**two** *separate copies*
T _main	
T _my_sub	

The linker joins globals by name; statics stay apart

Running It: Side Effects

./myprog

countG in my_sub is 2

resetS in my_sub is 0

countG is 2 not 1 **side effect**

reset is still 1 **no side effect**

main set countG to 1; my_sub changed it to 2

Why Globals Are Slow

```
sw   $2, %gp_rel(countG)($28)      countG = 1
jal  my_sub                        call
lw   $2, %gp_rel(countG)($28)      reload countG
```

The compiler must assume **any** *call it cannot see may change a global - so it reloads it*

Rust explicitly forces you to reload so you see the cost

Prefer local variables and const

The Same Program in Rust

main.rs

```
mod mysub ;
pub static COUNT_G : AtomicI32 = AtomicI32::new ( 0 ) ;
static RESET_S : AtomicI32 = AtomicI32::new ( 0 ) ;
```

mysub.rs

```
use crate::COUNT_G ;
static RESET_S : AtomicI32 = AtomicI32::new ( 0 ) ;
```

Two RESET_S - one per module, like C's static

The volatile Attribute

Tells the compiler the variable could **change at any time**

The compiler will not optimize its reads or writes away

Declared in C and C++ with **volatile**

When You Need volatile

In practice, only three kinds of variable:

1. *Memory-mapped* **peripheral registers**
2. *Globals changed by an* **interrupt** *routine*
3. *Globals shared by* **multiple tasks** *in a multi-threaded program*

Register Without volatile

```
uint8_t *my_reg = ( uint8_t * ) 0xC000 ;

// wait for the register to read non-zero
do {
} while ( ! ( *my_reg ) ) ;
```

*The optimizer reads *my_reg* **once**, *before the loop. It sees nothing that could change it - so the loop never ends*

Register With volatile

*uint8_t volatile *my_reg = (uint8_t *) 0xC000 ;*

do {
*} while (! (*my_reg)) ;*

Now every pass **re-reads** *the register*

The **data** *is volatile, not the pointer*

Interrupt Flag Without volatile

```
bool gb_etx_foundG = false ; //assignment not needed

main:           while ( !gb_etx_foundG ) {  }

interrupt:      if ( ETX == rx_char )
                        gb_etx_foundG = true ;
```

No call in the loop - the optimizer may load the flag once and loop forever

Interrupt Flag With volatile

```
bool volatile gb_etx_foundG = false ;

main:           while ( !gb_etx_foundG ) {  }

interrupt:      if ( ETX == rx_char )
                        gb_etx_foundG = true ;
```

volatile guarantees the compiler reads the flag on every pass

volatile in Rust

Rust has **no** *volatile keyword*

Registers: explicit volatile access

```
let my_reg = 0xC000 as *const u8 ;
while unsafe { my_reg.read_volatile ( ) } == 0 { }
```

Embedded register crates wrap this for you

Interrupt Flags in Rust

static ETX_FOUND_G : AtomicBool = AtomicBool::new (false) ;

main: *while !ETX_FOUND_G.load (Ordering::Acquire) { }*

interrupt: *ETX_FOUND_G.store (true, Ordering::Release) ;*

An atomic is both safe and never optimized away

volatile Is Not Enough for Threads

volatile stops the compiler removing accesses

It does **not** *make x = x + 1 indivisible*

Two threads can still lose an update

Use atomics for shared data:

C: <stdatomic.h>   C++: std::atomic

Rust: AtomicI32, AtomicBool, ...

Scope and Lifetime Together

Kind	Scope	Lifetime	Lives in
auto (local)	*the block*	*the call*	*stack*
local static	*the function*	*whole program*	*data*
file static	*the file*	*whole program*	*data*
global	*every file (extern)*	*whole program*	*data*
heap	*via a pointer*	*until freed*	*heap*

Across the Three Languages

	C	C++	Rust
global	*int xG ;*	*int xG ; or namespace*	*pub static (atomic)*
file private	*static int xS ;*	*namespace { }*	*default (no pub)*
local static	*static int xL ;*	*static int xL ;*	*static (atomic)*
declare	*extern*	*extern*	*mod + use*
heap	*malloc / free*	*vector, unique_ptr*	*Vec, Box*
volatile	*volatile*	*volatile*	*read_volatile*

Good Habits

Prefer **local** *variables and parameters*

Give every name the **smallest** *scope that works*

Make file-only data **static** *(C) or private (Rust)*

Mark names: **G** *global,* **S** *file static,* **L** *local static*

Use **const** *for anything that should not change*

What We Can Now Do

Scope	*where a name is visible*
Lifetime	*how long its memory exists*
Regions	*data segment, stack, heap*
Keywords	*extern static volatile pub*

Let's practice

Exercise: Using **scopepractice**